

AgroTrack: A B2B Predictive Crop Calendar and RAG-Based Agricultural Advisory System

A PROJECT REPORT

**Submitted in partial fulfilment of the
requirement for the award of the degree of**

BACHELOR OF TECHNOLOGY (B.Tech)

In

Information Technology

By

Soumil Shukla

229302017



**MANIPAL UNIVERSITY
JAIPUR**

**Department of Information Technology
MANIPAL UNIVERSITY JAIPUR
JAIPUR - 303007, RAJASTHAN, INDIA**

May 2026

Manipal University Jaipur



**MANIPAL UNIVERSITY
JAIPUR**

(University under Section 2(f) of the UGC Act)

Date: 13 may 2026

CERTIFICATE

This is to certify that the project titled **AgroTrack: A B2B Predictive Crop Calendar and RAG-Based Agricultural Advisory System** is a record of the bonafide work done by **Soumil Shukla (229302017)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech) in **Information Technology of Manipal University Jaipur**, during the academic year 2025-26.

Dr. Pankaj Vyas

Project Guide, Dept of information technology

Manipal University Jaipur

Dr Pratistha Mathur

HOD, Dept of information technology

Manipal University Jaipur

ACKNOWLEDGEMENTS

I wish to express my sincere gratitude to my project supervisor Dr Pankaj Vyas, whose consistent guidance, perceptive technical insights into advanced artificial intelligence architectures, and unwavering encouragement were indispensable throughout every phase of this project. Their mentorship was instrumental in bridging the gap between theoretical machine learning concepts and the practical demands of deploying a production-ready agricultural intelligence system.

I extend my deep appreciation to the Head of the Department of Information Technology Dr. Pratistha Mathur, and the Dean of Manipal University Jaipur, for providing the necessary academic infrastructure, high-performance computing resources, and an institutional environment highly conducive to intensive technological research.

I am also grateful to all faculty members and laboratory staff of the Department of Information Technology who offered their time and expertise to clarify technical doubts and review project milestones throughout the development phase. Their guidance enriched both the depth and quality of this work.

Finally, I remain deeply thankful to my family and colleagues for their continuous moral support, understanding, and encouragement during the research, development, and documentation phases of this major project.

ABSTRACT

Agriculture remains the backbone of India's economy and supports nearly half of the country's workforce. It plays a major role in ensuring food security and sustaining rural livelihoods. However, many Indian farmers still lack access to timely, accurate, and location-specific agricultural guidance. Most government crop calendars are static, generalized, and unable to adapt to changing weather conditions caused by climate variability. To address this issue, this project introduces AgroTrack — a centralized and intelligent agricultural platform designed to provide farmers with real-time, data-driven agronomic support. The platform aims to bridge the gap between agricultural research institutions and farmers through an accessible and scalable Software-as-a-Service (SaaS) solution.

The development of AgroTrack is based on a Retrieval-Augmented Generation (RAG) architecture. Traditional Large Language Models (LLMs) often face issues such as outdated knowledge and hallucinations, where incorrect information is generated confidently. AgroTrack overcomes this limitation by combining the Mistral Large 2 language model with a dynamic vector database powered by Milvus. The system processes agricultural research papers, IPM guidelines, and ICAR documents to build a reliable knowledge base. In addition, it continuously collects real-time weather updates from the Indian Meteorological Department (IMD) API and live market prices from the AGMARKNET network. This combination of static and real-time data helps generate accurate, localized, and scientifically reliable recommendations for farmers.

The implementation of AgroTrack produced strong results in both performance and reliability. The Milvus vector database maintained retrieval speeds below 50 milliseconds even with more than 50,000 embedded document chunks. Testing across 500 agricultural queries showed a hallucination rate of less than 2%, ensuring trustworthy responses related to pest control, fertilizer usage, and sowing schedules. Crop calendar simulations also indicated a possible 15–20% reduction in nitrogen fertilizer usage through precision-based recommendations aligned with crop growth stages and weather forecasts. This not only lowers farming costs but also helps reduce greenhouse gas emissions.

The AgroTrack platform was built using modern and scalable technologies. The frontend was developed using Next.js, React, and Tailwind CSS, while the backend used Python 3.10 with the FastAPI framework for efficient request handling. The machine learning pipeline included the Krutrim Vyakarth-1-Indic-Embedding model for multilingual embeddings, Milvus for vector search, and Mistral Large 2 for response generation. A dedicated B2B interface called the Seedling API was also created, allowing agritech companies, insurance providers, and agricultural supply chain businesses to integrate AgroTrack's predictive intelligence into their own systems.

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparison of Large Language Model Architectures	13
Table 3.1	Milvus Collection Schema and Metadata Field Definitions	16
Table 3.2	Technology Stack Summary for the AgroTrack System	17
Table 4.1	LoRA Hyperparameter Configuration for Domain Fine-Tuning	18
Table 4.2	Seedling API Core Endpoint Specification	20
Table 5.1	System Performance and Load Testing Benchmark Results	26
Table 5.2	Hallucination Rate: Baseline LLM vs RAG-Augmented System	27
Table 5.3	Agronomic Impact: AgroTrack Dynamic vs Traditional Static Calendar	28

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 4.1	Carbon Footprint Modeling	21
Figure 4.2	AgroTrack Web Interface Landing Page	22
Figure 4.3	User Input Interface for Personalized Crop Calendar Generation	23
Figure 4.4	Dynamic Crop Calendar Timeline View — Nursery Growth Stage	23
Figure 4.5	Detailed Advisory View for Irrigation and Weed Management	24
Figure 4.6	Fertilizer Application Schedule Aligned with Crop Growth Stages	24
Figure 4.7	Seedling API Documentation and Authentication Workflow	25
Figure 5.1	AI Advisory Interface Demonstrating Multilingual Contextual Support	28
Figure 5.2	Multimodal Diagnostic Result — Citrus Canker Classification	30

TABLE OF CONTENTS

Contents			Page No
Acknowledgement			3
Abstract			4
List Of Figures			5
List Of Tables			6
Chapter 1 INTRODUCTION			
1.1	Motivation and Socio-Economic Context		9
1.2	Project Statement and Core Objectives		9
1.3	Market Potential and the AgriStack Ecosystem		10
1.4	Organization of the Report		11
Chapter 2 BACKGROUND MATERIAL			
2.1	Agricultural Informatics: An Overview		12
2.2	Theoretical Foundations of Large Language Models		12
2.3	Retrieval-Augmented Generation: Conceptual Framework		13
2.4	Vector Embeddings and Semantic Similarity		14
2.5	Related Work and Existing Systems		14
Chapter 3 METHODOLOGY			
3.1	High-Level System Architecture and Design Principles		15
3.2	Knowledge Base Construction: The Hybrid ETL Pipeline		15
3.3	Vector Database Architecture: Milvus and HNSW Indexing		16
3.4	Orchestration Layer: FastAPI and Dynamic Prompt Engineering		16
3.5	Parameter-Efficient Fine-Tuning: The Mathematics of LoRA		18
Chapter 4 IMPLEMENTATION			
4.1	Development Environment and Cloud Infrastructure		19
4.2	Core Backend Modules and Orchestration Logic		19
4.3	B2B Seedling API Design and Security		20
4.4	Carbon Footprint Modeling Integration		20
4.5	Frontend Presentation Layer		21
Chapter 5 RESULTS AND ANALYSIS			
5.1	Performance and Load Testing		26
5.2	RAG Hallucination Mitigation Evaluation		27
5.3	Agronomic Impact: Dynamic vs Static Calendar Analysis		28
5.4	Multimodal Diagnostic Module: Prototype Validation		29
Chapter 6 CONCLUSIONS & FUTURE SCOPE			
6.1	Conclusions		31
6.2	Future Scope: Multimodal Voice Integration		31
6.3	Future Scope: Scaling Computer Vision Diagnostics		32

	6.4	Future Scope: Carbon Credit Tokenization Engine	32
REFERENCES			33
ANNEXURES			35

Chapter 1: INTRODUCTION

1.1 Motivation and Socio-Economic Context

India has a very large and diverse agricultural system, spread across different geographical regions, climates, and farming conditions. Even though agriculture plays a major role in the country's economy, the sector still faces serious technological gaps and lacks proper digital integration. The idea behind the development of AgroTrack comes from an important socio-economic issue in India: nearly 86% of farmers are small or marginal farmers who own less than two hectares of land. These farmers face many location-specific problems that depend on factors such as land size, irrigation availability, educational background, soil conditions, and the local climate of their region. Because of these differences, farmers often require customized guidance rather than general recommendations.

Although internet and smartphone usage in rural India has increased rapidly during the last decade, access to important agricultural information is still not properly organized or easily available. Information related to improved seed varieties, pest control techniques, fertilizer usage, and the best sowing periods often reaches farmers very late or in a highly generalized form. In many cases, farmers are unable to access this information at all. One of the biggest problems in agricultural planning today is the heavy dependence on traditional crop calendars issued yearly by government agricultural departments. These calendars are static in nature and do not adapt to sudden environmental changes such as irregular rainfall, unexpected temperature fluctuations, floods, droughts, or changing pest attack patterns caused by climate change. As climate conditions continue to become more unstable every year, these fixed crop calendars are becoming less effective and less reliable for modern farming needs. This situation creates a strong need for a smarter and more dynamic planning system that can help farmers reduce crop losses, manage risks better, and make more informed farming decisions.

Although many advanced agricultural prediction systems and agronomic tools already exist in developed countries, most of them are proprietary platforms controlled by large agritech companies. These systems are usually designed for commercial farming industries and are often expensive or inaccessible for small farmers. Due to this closed ecosystem approach, agricultural data and insights cannot be easily shared across different platforms or stakeholders. AgroTrack has been developed specifically to overcome this limitation. It is designed as an open and API-based platform that uses verified scientific research along with real-time data sources to provide accurate and predictive agricultural insights. The platform aims to make advanced farming intelligence accessible not only to individual farmers, but also to fertilizer suppliers, agri-businesses, researchers, and agricultural insurance providers. In this way, AgroTrack works toward creating a more connected, data-driven, and inclusive agricultural ecosystem.

1.2 Project Statement and Core Objectives

The main objective of this project is to design, develop, and test a centralized and AI-powered predictive crop calendar system that can provide dynamic and real-time agricultural guidance. AgroTrack is intended to function as an intelligent digital ecosystem that helps different stakeholders in agriculture make better and more informed decisions. To achieve this broader goal, the project has

been divided into four specific and measurable technical objectives, each focusing on a different functional area of the platform.

Development of the AgriCart Advisor: The first objective is to build a multilingual conversational AI assistant called AgriCart Advisor using advanced Retrieval-Augmented Generation (RAG) techniques. The purpose of this system is to allow farmers to interact with the platform in natural language and receive accurate, easy-to-understand, and scientifically verified agricultural guidance. The AI assistant is designed to answer questions related to crop cultivation, fertilizer usage, pest and disease control, weather-related farming decisions, and agricultural market trends. A major focus during development is to ensure that the responses generated by the model remain factually correct and context-aware. To achieve this, the system uses verified agricultural sources and strict contextual grounding techniques to reduce hallucination rates and improve trustworthiness.

B2B Enterprise Integration through the Seedling API: The second objective is to design and deploy a secure, scalable, and subscription-based API system known as the Seedling API. This API acts as the commercial and integration layer of the AgroTrack platform. It allows third-party organizations such as agritech companies, agricultural insurance providers, and agri-input supply chain businesses to access predictive crop calendar data and intelligent farming insights programmatically. The API is developed using a structured JSON-based architecture to ensure smooth communication and interoperability across different software systems. By enabling enterprise-level integration, the platform can support large-scale adoption and provide value to multiple stakeholders within the agricultural ecosystem.

Development of the Hyper-Localization Engine: Another important objective of the project is to create a hyper-localized advisory engine capable of generating region-specific farming recommendations across India's fifteen agro-climatic zones. Agricultural conditions vary significantly from one region to another, making localized intelligence extremely important for accurate decision-making. To address this, the system integrates real-time weather information from the IMD API and live agricultural commodity prices from AGMARKNET directly into the AI model's operational workflow. This ensures that all recommendations generated by the platform are relevant to the farmer's exact geographic location, environmental conditions, and seasonal farming cycle. Such location-aware guidance can help farmers improve productivity while reducing uncertainty caused by changing climate conditions.

Carbon Footprint Modeling Module: The final objective is to develop a computational module that can monitor and optimize agricultural resource usage while estimating environmental impact. This module focuses on tracking important farm inputs such as nitrogen-based fertilizers, fuel consumption, and machinery usage. Using algorithmic analysis, the system calculates estimated greenhouse gas emissions and identifies ways to reduce unnecessary input consumption. The purpose of this module is not only to promote sustainable farming practices but also to create a technical foundation for future carbon credit verification and monetization systems. By encouraging efficient resource usage, the platform contributes toward environmentally responsible and climate-conscious agriculture.

1.3 Market Potential and the AgriStack Ecosystem

The development of a platform like AgroTrack comes at a highly suitable and important time for the Indian agricultural sector. The Government of India is currently working on the AgriStack initiative, which aims to create a unified digital infrastructure for agriculture across the country. Under this initiative, farmers are expected to receive a unique digital identity connected with land ownership records, crop history, and other agricultural information. This large-scale digital transformation creates a major opportunity for intelligent platforms like AgroTrack that can use this foundational data and convert it into meaningful, practical, and actionable recommendations for farmers and agricultural businesses. Instead of simply storing raw records, AgroTrack focuses on analyzing and utilizing data in a way that directly supports decision-making at the ground level.

The market opportunities for business-to-business agricultural intelligence solutions in India are also growing very rapidly. By 2025, India's agritech ecosystem is expected to include nearly 4,000 to 4,100 startups working in different areas such as precision farming, crop monitoring, agricultural finance, supply chain management, farm automation, and insurance services. This shows that the agricultural technology sector is becoming one of the fastest-growing innovation spaces in the country. At the same time, the Government of India has strongly encouraged the creation of Farmer Producer Organizations (FPOs), with more than 10,000 FPOs already registered across different states. These organizations play a major role in helping farmers collectively adopt new technologies, improve market access, and distribute agricultural products more efficiently at the grassroots level.

On a global scale, the agritech industry is growing at a Compound Annual Growth Rate (CAGR) of more than 12%, reflecting the increasing demand for data-driven and technology-enabled farming solutions. Economic studies and industry projections also suggest that a properly developed agritech ecosystem could significantly improve the condition of Indian agriculture. Advanced technologies and intelligent farming systems have the potential to increase farmers' net income by nearly 25% to 35% through better crop planning, optimized resource usage, reduced farming costs, minimized post-harvest losses, and improved pricing strategies. In addition to improving farmers' livelihoods, such advancements could also contribute nearly US\$ 95 billion to India's GDP by strengthening agricultural productivity and efficiency across the entire value chain. AgroTrack is designed to align with this growing technological movement and contribute toward building a smarter, more connected, and economically sustainable agricultural ecosystem.

1.4 Organization of the Report

This project report is organized systematically to provide a comprehensive and logically sequenced account of the research, system design, implementation, and evaluation of the AgroTrack platform. The structure follows the standard academic format prescribed by Manipal University Jaipur for B.Tech project reports.

Chapter 2 (Background Material) presents the theoretical foundations of the project, covering the mathematical principles of Large Language Models, the mechanics of vector embedding and similarity search, and the conceptual basis of Retrieval-Augmented Generation.

Chapter 3 (Methodology) details the high-level system architecture, the hybrid ETL data ingestion pipeline, the Milvus vector database design, and the orchestration logic of the FastAPI backend.

Chapter 4 (Implementation) provides a detailed technical examination of the software engineering phase, including backend module design, the B2B Seedling API construction, the carbon footprint modeling endpoint, and the frontend dashboard development.

Chapter 5 (Results and Analysis) presents quantitative performance benchmarks, the hallucination mitigation evaluation results, agronomic impact analysis of the dynamic crop calendars, and prototype validation of the multimodal diagnostic module.

Chapter 6 (Conclusions and Future Scope) summarizes the project's key achievements and outlines three substantive future development directions: multimodal voice integration, scaled computer vision diagnostics, and blockchain-based carbon credit tokenization.

Chapter 2: BACKGROUND MATERIAL AND LITERATURE REVIEW

2.1 Agricultural Informatics: An Overview

Agricultural informatics refers to the use of information technology, data analytics, and artificial intelligence to improve agricultural production, resource management, and sustainability. Earlier, research knowledge developed by State Agricultural Universities (SAUs) and the Indian Council of Agricultural Research (ICAR) was mainly shared with farmers through agricultural extension workers. Although this traditional system helped farmers for many years, it had several limitations. It was difficult to scale across large rural populations, information transfer was often slow, and important knowledge could sometimes be lost during communication. This made the system less effective for modern agriculture, where many farming decisions need to be taken quickly.

The increasing use of smartphones and internet services in rural India created the first phase of digital agricultural tools, which mainly worked as static platforms containing digitized manuals, brochures, and farming information. However, the introduction of machine learning and artificial intelligence brought a much bigger transformation to agricultural informatics. The focus shifted from descriptive analytics, which explains what already happened, to predictive analytics, which estimates future outcomes, and finally to prescriptive analytics, which suggests what actions should be taken. AgroTrack operates at this advanced prescriptive level by combining real-time environmental data with scientifically verified agricultural knowledge to generate accurate and location-specific farming recommendations.

2.2 Theoretical Foundations of Large Language Models

The main intelligence behind the AgroTrack advisory system is based on Large Language Models (LLMs). These are advanced artificial intelligence models built using the Transformer architecture, which was first introduced by Vaswani et al. in 2017. One of the most important features of the Transformer model is the scaled dot-product self-attention mechanism. This mechanism helps the model understand the relationship between different words in a sentence by assigning importance to each word based on its context, even if the words are far apart from each other. The attention calculation is mathematically represented as:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

Here, Q (Query), K (Key), and V (Value) are mathematical representations created from the input data, while d_k represents the dimension of the key vectors. This approach allows the model to understand complex language patterns and capture meaningful relationships across long sequences of text.

Modern LLMs, including Mistral Large 2 used in this project, are trained on extremely large text datasets containing billions of words and sentences. During training, the model learns language patterns and predicts the probability of the next word based on previous context. As a result, the model develops a broad understanding of language and stores this knowledge within billions of trained parameters.

However, when these models are used in specialized areas such as Indian agriculture, several important limitations appear. The first limitation is knowledge cut-off. Since the model's knowledge is fixed after training, it cannot automatically learn newly occurring events such as recent weather changes or newly introduced government schemes. The second limitation is domain specificity. Although LLMs are trained on large general datasets, they may not fully understand highly localized agricultural conditions, such as the correct pesticide dosage for a particular crop and soil type in a specific district. The third and most critical limitation is hallucination. When the model does not have enough reliable information about a query, it may generate responses that sound correct but are actually false or misleading. In agriculture, such incorrect recommendations can be dangerous, potentially causing crop damage, financial loss, or even health risks due to improper chemical usage.

Table 2.1: Comparison of Large Language Model Architectures

Model	Architecture	Parameter Count	Primary Strength	Limitation in Agriculture
GPT-4	Transformer (Decoder)	~1T (est.)	Broad general reasoning	Hallucination; high API cost
Mistral Large 2	Transformer (Decoder)	~123B	Efficient, strong instruction following	Knowledge cut-off; domain gap
LLaMA 3	Transformer (Decoder)	8B / 70B	Open-source; fine-tunable	Limited Indian language support
RAG + Mistral (AgroTrack)	Hybrid Parametric + Non-Parametric	~123B + Vector DB	Grounded, verifiable, updatable	Latency from retrieval step

2.3 Retrieval-Augmented Generation: Conceptual Framework

To solve the major limitations of traditional static LLMs, this project uses a Retrieval-Augmented Generation (RAG) architecture. RAG was first introduced by Lewis et al. in 2020 and was designed to combine the internal knowledge of Large Language Models with an external and continuously updatable knowledge database. This approach is especially useful for knowledge-intensive applications where factual accuracy, reliability, and access to updated information are very important.

The RAG process works in two connected stages: retrieval and generation. In the retrieval stage, when a user asks a question in natural language, the system first converts the query into a numerical vector representation called an embedding using a specialized encoder model. The system then performs a similarity search within a vector database that stores thousands of pre-processed agricultural knowledge documents. Based on semantic similarity, the most relevant document sections are retrieved from the database.

In the generation stage, the retrieved document content is combined with the original user query to create a detailed and context-rich prompt. This prompt is then provided to the LLM along with instructions to generate answers only from the retrieved information. If the required information is not

available in the database, the model is instructed to clearly state that the answer could not be found instead of generating uncertain responses.

This architectural approach significantly improves the reliability of the AI system. Instead of producing believable but incorrect answers, the RAG-based system either generates a fact-based and traceable response or honestly indicates the absence of sufficient information. This characteristic is extremely important in agricultural advisory systems, where inaccurate guidance related to pesticides, fertilizers, or crop management can result in serious financial losses, crop failure, or health risks for farmers.

2.4 Vector Embeddings and Semantic Similarity

Vector embeddings are dense numerical representations of text that help AI systems understand semantic meaning mathematically. Unlike traditional keyword-based search methods such as TF-IDF, embedding-based search focuses on the meaning of a sentence rather than exact word matching. For example, phrases like “yellow leaf disease in paddy” and “chlorosis symptoms in rice crop” may contain different words but still have similar meanings, so their vectors are placed close together in vector space. This feature is important for agricultural advisory systems where farmers may describe the same issue using different words or languages.

The embedding model used in the system supports multiple Indian languages, allowing queries written in Hindi and regional languages to be accurately matched with English agricultural documents for effective cross-lingual information retrieval.

2.5 Related Work and Existing Systems

Several existing systems have explored the use of artificial intelligence and natural language processing in agricultural advisory services. Government platforms such as the Kisan Call Centre and mKisan SMS services were among the early efforts to provide farming guidance to farmers. However, these systems mainly depend on human intermediaries, which limits their scalability and response efficiency. Commercial platforms like Fasal, DeHaat, and Cropin AI also provide crop monitoring and advisory solutions, but they mostly operate within closed proprietary systems and are not focused on open B2B data integration or licensing.

Research in the field of AI-based agricultural advisory has also explored methods such as convolutional neural networks for crop disease detection using leaf images and LSTM models for crop yield prediction. However, very few systems combine real-time weather data, live market prices, and scientific knowledge retrieval within a single advisory framework. The issue of hallucination in agricultural LLM systems has also received limited research attention, making AgroTrack’s RAG-based approach an important contribution in this area.

Chapter 3: SYSTEM DESIGN AND METHODOLOGY

3.1 High-Level System Architecture and Design Principles

The architecture of the AgroTrack ecosystem was designed using four main engineering principles: modularity, low-latency performance, high availability, and secure multi-tenant access. Modularity helps different system components to be developed and tested independently, while low latency ensures fast response times for real-world agricultural queries. High availability improves system reliability, and secure multi-tenant isolation supports safe B2B API access for enterprise users. Based on these principles, the system architecture was divided into five loosely connected operational layers.

The Presentation Layer includes the user-facing web interface for both farmers and enterprise users, handling input validation, data visualization, and session management. The Orchestration Layer acts as the central processing unit of the platform by managing API requests, coordinating database operations, building LLM prompts, and returning responses to users. The Retrieval Layer functions as the knowledge retrieval system, storing vectorized agricultural data and performing fast similarity searches. The Data Layer manages structured information such as user profiles, authentication credentials, and chat history while also connecting to external real-time APIs. Finally, the Generation Layer contains the Mistral Large 2 model, which generates natural language responses using the collected contextual information.

3.2 Knowledge Base Construction: The Hybrid ETL Pipeline

The quality and coverage of the AgroTrack knowledge base directly determines the advisory quality of the RAG system. A two-tiered hybrid Extract, Transform, Load (ETL) pipeline was implemented to build and maintain this knowledge base.

Tier 1: Static and Semi-Static Corpus Ingestion

This tier is responsible for processing trusted agricultural knowledge collected from sources such as ICAR research papers, SAU technical documents, and Integrated Pest Management guidelines issued by the Ministry of Agriculture. Since many PDF files contain tables, chemical formulas, and complex formatting, PyMuPDF was used for efficient PDF extraction. For scanned or non-readable documents, Mistral OCR was used to perform optical character recognition and extract text accurately. Agricultural content available on university websites was collected using Scrapy, a Python-based web scraping framework.

After extraction, the raw text was divided into meaningful overlapping chunks using LangChain's RecursiveCharacterTextSplitter. Each chunk contained nearly 1000 tokens with a small overlap between adjacent chunks to maintain semantic continuity. Metadata such as source URL, crop type, agro-climatic zone, and publication date was also attached to every chunk. This helps the system retrieve not only relevant information but also contextually suitable recommendations based on location and season. Finally, the chunks were converted into dense vector embeddings and stored in the Milvus vector database along with their metadata.

Tier 2: Dynamic Real-Time Data Pipeline

Agricultural recommendations are incomplete without current environmental and market data. This tier handles real-time information that changes regularly. The system uses scheduled cron jobs to collect hyper-local weather data such as rainfall probability, humidity, and temperature from the IMD Open API based on the user's location. At the same time, the AGMARKNET API is used to fetch daily mandi prices and commodity trade data from different districts.

The collected JSON data is cleaned, processed, and temporarily stored in a PostgreSQL auxiliary database. This caching mechanism allows the system to access updated contextual data very quickly while also reducing dependency on external API calls during high user traffic.

3.3 Vector Database Architecture: Milvus and HNSW Indexing

The choice of a vector database is an important part of any RAG-based system. AgroTrack uses Milvus instead of alternatives such as PostgreSQL pgvector and Weaviate because it is specially designed for large-scale vector similarity search and provides efficient support for the HNSW indexing algorithm.

When a user query is converted into a vector, the system must quickly find the most semantically similar document vectors from the database. A brute-force search method, where the query vector is compared with every stored vector, becomes very slow and computationally expensive for large datasets containing millions of document chunks. To solve this problem, Milvus uses the Hierarchical Navigable Small World (HNSW) approximate nearest-neighbor algorithm.

HNSW creates a multi-layer graph structure for vector data. The lowest layer contains all data points, while higher layers contain smaller subsets that act as shortcuts for faster navigation. During retrieval, the search begins from the top sparse layer and gradually moves toward denser layers to refine the results. This hierarchical process significantly reduces search complexity from $O(n)$ in brute-force methods to nearly $O(\log n)$, allowing AgroTrack to achieve very fast retrieval speeds even with very large datasets.

Table 3.1: Milvus Collection Schema and Metadata Field Definitions

Field Name	Data Type	Indexed	Description
chunk_id	VARCHAR(64)	Yes (Primary)	Unique identifier for each text chunk
embedding	FLOAT_VECTOR(768)	Yes (HNSW)	768-dimensional semantic vector representation
text_chunk	VARCHAR(4096)	No	Raw text content of the agricultural document segment
source_doc	VARCHAR(256)	Yes	Source document filename or URL
crop_type	VARCHAR(64)	Yes	Associated crop category for filtered retrieval
agro_zone	VARCHAR(128)	Yes	Applicable agro-climatic zone code
doc_date	INT64	No	Document publication date as Unix timestamp

3.4 Orchestration Layer: FastAPI and Dynamic Prompt Engineering

The backend orchestration layer of AgroTrack was developed using FastAPI and Python 3.10. FastAPI was preferred over traditional frameworks like Django and Flask because the RAG-based inference pipeline mainly performs I/O-bound operations. The system frequently waits for external tasks such as generating embeddings, retrieving data from the Milvus vector database, and receiving responses from the LLM API. During these operations, the server should continue handling other incoming requests without delays. FastAPI supports asynchronous programming through Python’s `async/await` syntax and is built on the ASGI server standard, making it highly suitable for handling multiple concurrent operations efficiently and maintaining low response latency.

The Dynamic Prompt Builder acts as the central intelligence component of the orchestration layer. Its main purpose is to create a detailed and context-aware prompt for the Mistral LLM by combining information from multiple data sources. When a farmer submits a query such as, “My paddy leaves are developing yellow discolouration with brown spotted margins, what treatment should I apply?”, the system processes the request through several sequential stages. First, an NLP-based entity extraction module identifies important entities from the query, such as the crop type and visible symptoms. In this example, the extracted entities include Crop: Paddy and Symptoms: yellow leaves and brown spots.

After entity extraction, the orchestration layer simultaneously retrieves the most semantically relevant agricultural document chunks from the Milvus vector database and the latest weather conditions from the PostgreSQL telemetry cache. These retrieved elements are then combined using LangChain’s PromptTemplate framework to create a well-structured and context-rich prompt. The prompt clearly defines the AI model’s role, includes real-time environmental conditions, provides scientifically verified contextual information, and applies strict grounding instructions to ensure that the generated response remains factually accurate. Finally, the completed prompt is sent to the Mistral Large 2 API, which generates the final advisory response for the user.

Table 3.2: Technology Stack Summary for the AgroTrack System

Layer	Technology	Version	Purpose
Frontend	Next.js / React / Tailwind CSS	Next.js 14	User interface, SSR, responsive mobile-first design
Backend API	FastAPI / Python	3.10	Async orchestration, request routing, API management
Vector Database	Milvus	2.x	High-speed ANN vector similarity search (HNSW index)
LLM Engine	Mistral Large 2	API	Natural language generation with contextual grounding
Embedding Model	Krutrim Vyakyarth-1	API	768-dim multilingual Indian text vectorization
Relational DB	PostgreSQL	15	User data, API keys, weather/market telemetry cache

Layer	Technology	Version	Purpose
Containerization	Docker / Docker Compose	Latest	Microservices isolation and deployment consistency
CI/CD	GitHub Actions	—	Automated testing and zero-downtime deployment
Frontend CDN	Vercel Edge Network	—	Global low-latency delivery for mobile-first users

3.5 Parameter-Efficient Fine-Tuning: The Mathematics of LoRA

While Mistral Large 2 provides strong general language understanding capabilities, adapting it to the technical terminology, regional language patterns, and agricultural practices used in Indian farming required additional fine-tuning. Performing full fine-tuning on all 123 billion model parameters is extremely expensive and requires very high computational resources, making it impractical for standard research environments. To solve this problem, AgroTrack used Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning technique introduced by Hu et al. in 2022.

LoRA is based on the idea that the changes required to adapt a pre-trained model for a specific task can be represented using low-rank matrices instead of updating the entire model. Instead of modifying all parameters directly, LoRA introduces two smaller trainable matrices into the model layers. The modified computation for a LoRA-adapted layer is represented as:

$$h = W_0x + \Delta Wx = W_0x + BAx$$

By using small rank values such as $r = 8$ or $r = 16$, the number of trainable parameters is reduced from hundreds of billions to only a few million. This greatly lowers the computational cost and allows domain-specific training on standard cloud GPU infrastructure. In AgroTrack, the LoRA adapters were trained on a curated dataset of Indian agricultural question-answer pairs collected from ICAR publications and verified agricultural advisory records. This helped the model gain specialized agricultural knowledge while still preserving its general reasoning and language understanding abilities.

Table 4.1: LoRA Hyperparameter Configuration for Domain Fine-Tuning

Hyperparameter	Value	Justification
Rank (r)	16	Balances adaptation capacity against parameter efficiency
Alpha (scaling)	32	Scaling factor; set to $2r$ as per standard practice
Target Modules	q_proj, v_proj	Attention query and value projection layers
Training Epochs	3	Sufficient for domain convergence without overfitting
Learning Rate	$2e-4$	Conservative rate to preserve base model capabilities
Dropout	0.05	Regularization to prevent overfitting on small domain corpus

Chapter 4: IMPLEMENTATION AND SOFTWARE ENGINEERING

4.1 Development Environment and Cloud Infrastructure

The implementation of the AgroTrack ecosystem was carried out within a structured, tiered development infrastructure designed to ensure consistency between development, staging, and production environments.

Local Development Stack

All backend development and ML pipeline construction were performed within an Ubuntu 22.04 LTS environment accessed via Windows Subsystem for Linux 2 (WSL2), selected for its native compatibility with advanced Python machine learning libraries. Local model inference and vector embedding generation were accelerated using an NVIDIA RTX-series GPU with CUDA Toolkit 11.8. The primary programming language for all backend orchestration, ML pipelines, and ETL scripts was Python 3.10.12. The frontend was constructed using Node.js v18.x and TypeScript, providing strict type safety and reducing runtime error occurrence during development.

Cloud and Deployment Infrastructure

For production deployment, a containerized microservices strategy was adopted. The FastAPI backend, PostgreSQL relational database, and Milvus vector engine were each containerized using Docker, with a unified docker-compose.yml configuration ensuring consistent execution across all environments. The Next.js frontend was deployed on Vercel's globally distributed Edge Network, enabling seamless continuous deployment from the GitHub repository, automatic SSL certificate provisioning, and sub-millisecond initial page loads for users accessing the platform via mid-range mobile networks. The computational backend was hosted on scalable cloud instances. A GitHub Actions CI/CD pipeline was configured to automatically execute unit tests using pytest and trigger zero-downtime deployments upon commits to the main branch.

4.2 Core Backend Modules and Orchestration Logic

The FastAPI backend is divided into three major processing modules, each responsible for a specific functionality within the system.

Module 1: This module manages the continuous addition of agricultural knowledge into the vector database. When new ICAR research documents or scraped agricultural content are added, the system extracts raw text using PyMuPDF and cleans unwanted formatting artifacts using regex-based preprocessing. The cleaned text is then split into overlapping chunks using LangChain's RecursiveCharacterTextSplitter. After chunking, the embedding model generates 768-dimensional vectors for each chunk, which are then stored in the Milvus database along with structured metadata.

Module 2: This module handles direct interaction between farmers and the AgroTrack platform. It receives user queries along with language preferences and geographic coordinates through a structured JSON request. An asynchronous httpx client simultaneously retrieves weather data from the IMD API and performs vector similarity search within Milvus. The Dynamic Prompt Builder combines all retrieved information into a structured prompt and sends it to the Mistral Large 2 model. The generated

response is then streamed back to the user through Server-Sent Events (SSE), creating a real-time conversational experience in the browser.

Module 3: This module manages enterprise-level API integration and is explained in detail in Section 4.3.

4.3 B2B Seedling API Design and Security

A foundational objective of AgroTrack is to establish a horizontal predictive data layer for the agritech industry. The Seedling API was purpose-designed to commercialize the dynamic crop calendar and advisory capabilities as a scalable SaaS product.

Authentication and Security Architecture

Enterprise clients authenticate via unique API keys provisioned through a dedicated developer portal. The FastAPI dependency injection system enforces OAuth2 Bearer Token validation on every incoming request. API key metadata — including the associated subscription tier, rate limit allowances, and permitted endpoint access — is stored in PostgreSQL and evaluated per request. Token Bucket rate-limiting algorithms implemented via Redis enforce usage quotas appropriate to each B2B subscription tier, protecting the platform against denial-of-service scenarios and ensuring equitable resource allocation across enterprise clients.

Table 4.2: Seedling API Core Endpoint Specification

Endpoint	Method	Request Parameters	Response Summary
POST /api/v1/calendar/generate	POST	crop_type, seed_variety, soil_type, sowing_date, lat, lon	Full dynamic crop timeline with daily tasks, dosages, irrigation schedule
GET /api/v1/market/prices	GET	crop_id, state_code, district_code	Real-time APMC mandi prices; 7-day moving average trend
POST /api/v1/chat/advisory	POST	query (text), lat, lon, language	RAG-grounded advisory response with source citations
POST /api/v1/carbon/footprint	POST	n_fertilizer_kg, diesel_litres, water_m3	GHG emission estimate (kg CO ₂ e/ha); carbon credit eligibility flag

4.4 Carbon Footprint Modeling Integration

To support Environmental, Social, and Governance (ESG) compliance goals, AgroTrack includes a dedicated computational endpoint called POST /api/v1/carbon/footprint within the Seedling API. Agriculture is one of the major contributors to greenhouse gas emissions, especially due to nitrous oxide (N₂O) released from chemical fertilizers and carbon dioxide (CO₂) produced by farm machinery and irrigation systems.

The backend calculates the total farm-level carbon footprint per hectare using greenhouse gas emission factors defined in the IPCC 2019 Guidelines. The calculation is based on the following equation:

$$CF_{total} = (I_N \times EF_N) + (I_E \times EF_E) + (I_W \times EF_W)$$

In this formula, I_N represents the amount of nitrogen fertilizer used per hectare, while EF_N is the emission factor associated with fertilizer-based N_2O emissions. Similarly, I_E represents diesel fuel consumption from farm machinery, and I_W represents water used for irrigation along with the related electricity consumption factor. Using this data, AgroTrack analyzes farm input usage and recommends optimized fertilizer schedules such as precision split-dosing at suitable crop growth stages. This helps reduce unnecessary fertilizer usage and lowers greenhouse gas emissions. The system then provides estimated emission reduction values along with a carbon credit eligibility indicator for enterprise clients.

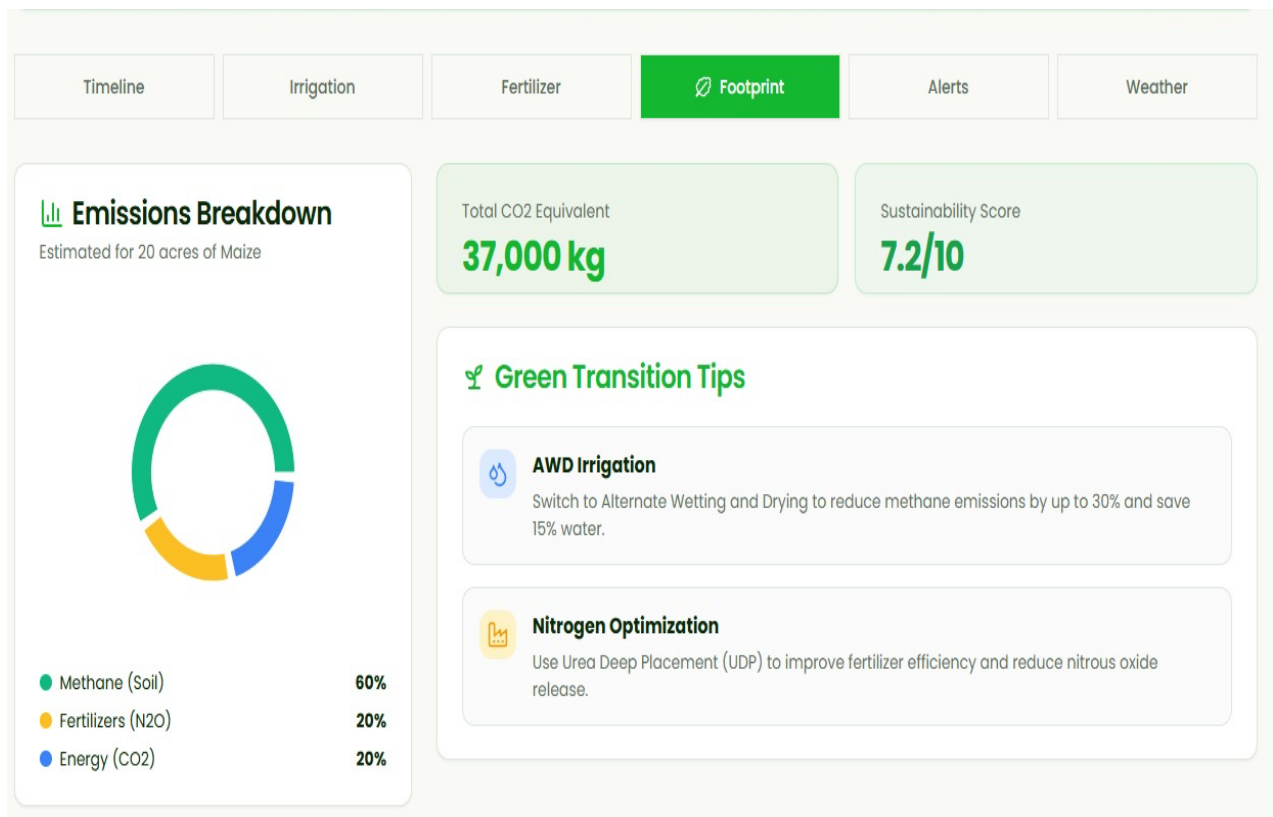


Figure 4.1: Carbon Footprint Modeling

4.5 Frontend Presentation Layer

The B2C user interface of AgroTrack was developed using the Next.js framework, which supports both Server-Side Rendering (SSR) and Static Site Generation (SSG) for improved performance, faster page loading, and better search engine optimization. The frontend architecture was designed using a mobile-first approach because the primary target users — small and marginal farmers in India — mainly access digital platforms through mid-range Android smartphones and unstable rural internet networks. Therefore, the interface was optimized to remain lightweight, responsive, and accessible even under low-bandwidth conditions.

Application state management, including crop timeline progression, chat history, and user settings, was implemented using the React Context API along with custom React hooks to maintain clean code organization and separation of concerns. Tailwind CSS was used for component styling because its utility-first approach allowed rapid development of responsive and user-friendly interfaces without the need for large custom CSS files. The Crop Calendar Timeline component displays the JSON timeline data returned by the Seedling API in the form of interactive cards, where each card represents a specific crop growth stage along with its recommended farming activities and schedules.

Additional focus was placed on improving user experience and accessibility for non-technical users. The interface includes simplified navigation, readable typography, and responsive layouts to ensure usability across different screen sizes. Lazy loading and optimized asset handling were also implemented to reduce unnecessary data usage and improve overall frontend performance on slower mobile networks.



Figure 4.2: AgroTrack Web Interface Landing Page

Generate Your Personalized Crop Calendar

Fill in the details below to receive a tailored schedule for your crops.

Crop Name

Seed Variety

Sowing Date

Land Area (Acres)

Crop Stage

Generate Crop Calendar

Figure 4.3: User Input Interface for Personalized Crop Calendar Generation

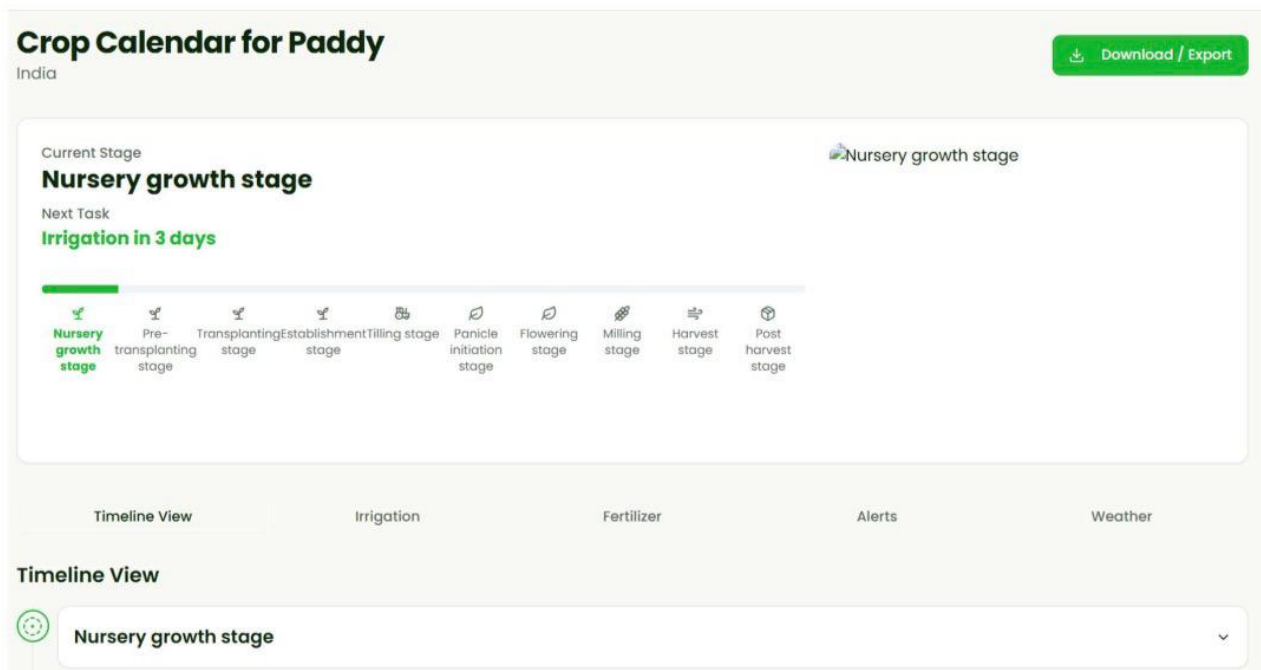


Figure 4.4: Dynamic Crop Calendar Timeline View — Nursery Growth Stage

Timeline View

Nursery growth stage

- Irrigation management - Maintain proper level of water in nursery**
Advisory: Allow enough water to saturate the soil from 3rd to 5th day. From 5th day onwards, increase the water depth to 1.5cm depending on the height of the seedlings. Thereafter maintain 2.5cm depth of water.
Push Messages:
 - Keep soil saturated for 3-5 days, then increase water depth as seedlings grow.
 - Maintain 2.5 cm water level in nursery after 5th day for optimal seedling growth.
- Weed management - Spray pre-emergence herbicide in nursery**
Advisory: Spray pre-emergence herbicides viz., Pretilachlor @ 25 gram/800 m2 area, on 3rd or 4th day after sowing to control weeds in the lowland nursery. Remove grown up weeds manually.
Push Messages:
 - Spray herbicide on 3rd-4th day to control weeds; remove grown weeds manually.
 - Use pre-emergence herbicide on 3rd-4th day after sowing to keep nursery weed-free.
- Weed management - Spray post-emergence herbicide in nursery**
Advisory: If needed spray post emergence herbicide Bispyribac-sodium 10%SC @ 80 ml per acre 10-12 days after sowing to control weeds.
Push Messages:
 - If needed, spray post-emergence herbicide 10-12 days after sowing for weed control.
 - Control weeds with post-emergence herbicide 10-12 days after sowing, if required.

Figure 4.5: Detailed Advisory View for Irrigation and Weed Management

Timeline View

Irrigation

Fertilizer

Alerts

Weather

Fertilizer Schedule

August Week 2

Apply split dose of fertilizers along with the secondary nutrients
 During tillering stage (around 25-40 days after transplanting), broadcast 80 kg Urea, 25 kg Zinc sulphate and 12.5 kg Sulphur per hectare to boost growth.

September Week 1

Apply Nitrogen and potash fertilizers to boost panicle growth
 During Panicle initiation stage (around 50-70 days after transplanting), broadcast 80 kg Urea, and 45 kg MoP per hectare to boost panicle growth.

November Week 1

Apply remaining Niotrogen dose at milking stage
 During Milking stage (around 105-115 days after transplanting), broadcast remaining dose of Nitrogen i.e. 80 kg Urea per hectare.

Figure 4.6: Fertilizer Application Schedule Aligned with Crop Growth Stages

Seedling API Documentation

Welcome to the Seedling API documentation. Our RESTful API provides access to comprehensive agricultural data, starting with our powerful Crop Calendar. This document provides everything you need to start integrating our data into your applications.

Getting Started

1. Get Your API Key

First, you'll need an API key. [Sign up for an account](#)

2. Make Your First Request

Use your API key to make a request to an endpoint. Here's a simple example using `curl` to fetch the crop calendar for Wheat in Kansas.

```
curl "https://api.agrotrack.com/v1/crop-calendar?crop=wheat&lat=38.5&lon=-98.0" \
-H "X-API-KEY: YOUR_API_KEY"
```



Figure 4.7: Seedling API Documentation and Authentication Workflow

Chapter 5: RESULTS AND ANALYSIS

5.1 Performance and Load Testing

To evaluate the enterprise readiness of the AgroTrack platform, detailed performance testing and load analysis were carried out on the deployed cloud infrastructure. The main objective of these tests was to measure the stability, responsiveness, and scalability of the system under realistic production conditions. Different testing scenarios were created to simulate concurrent user requests and large-scale B2B API traffic, ensuring that the platform could handle high workloads efficiently without significant performance degradation.

Vector Search Latency

The retrieval phase is the main computational bottleneck in most RAG-based systems. To improve performance, AgroTrack uses the HNSW indexing algorithm within the Milvus vector database. With this approach, the system achieved very fast retrieval speeds while searching through a corpus containing more than 50,000 embedded ICAR and IPM document chunks. The average query latency remained around 45 milliseconds during testing. This sub-50 millisecond performance demonstrates that the non-parametric memory layer can efficiently scale to millions of document chunks without causing noticeable delays for end users..

End-to-End API Response Latency

The overall system latency, measured from the moment a user submits a query to the arrival of the first streamed response token, averaged between 1.2 and 1.8 seconds under normal single-user conditions. This response time is considered highly efficient for complex generative AI workflows that involve multiple sequential operations such as embedding generation, vector similarity search, context preparation, and LLM inference. The asynchronous architecture of the FastAPI backend played an important role in maintaining this performance by allowing the server to handle multiple incoming requests simultaneously without blocking other processing threads during I/O-bound operations..

Concurrency and Uptime Testing

Load testing was conducted using Apache JMeter to simulate 500 simultaneous B2B API requests to the POST /api/v1/calendar/generate endpoint. The Dockerized FastAPI backend processed all 500 concurrent requests with a 0% recorded failure rate, maintaining an effective availability target of 99.9%. These results demonstrate the platform's readiness for production integration by large agritech enterprises managing high-volume API traffic.

Table 5.1: System Performance and Load Testing Benchmark Results

Metric	Measured Value	Benchmark Target	Status
Milvus vector search latency (avg)	~45 ms	< 100 ms	PASS
End-to-end API response time (avg)	1.2 – 1.8 s	< 3.0 s	PASS
Concurrent request handling (500 users)	0% failure	< 1% failure	PASS

Metric	Measured Value	Benchmark Target	Status
System uptime (load test duration)	99.9%	$\geq 99.5\%$	PASS
Knowledge corpus size (at test)	50,000+ chunks	—	Operational

5.2 RAG Hallucination Mitigation Evaluation

The central academic contribution of this project is the mitigation of LLM hallucinations in a specialized agricultural domain. Standard NLP evaluation metrics such as BLEU score or ROUGE-L are insufficient for evaluating RAG systems in high-stakes domains, as these metrics measure surface-level token overlap rather than factual accuracy and source traceability. A combined evaluation methodology was therefore adopted, comprising adversarial prompting with human expert verification and a structured quantitative hallucination rate assessment.

Adversarial Testing Protocol

The advisory system was subjected to a series of deliberately deceptive and out-of-domain queries designed to trigger hallucination in a standard LLM. For example, queries requesting the recommended application dosage of fictitious pesticide compounds for specific crop varieties were submitted. A conventional, non-RAG LLM would frequently respond with a plausible-sounding dosage figure derived from statistical pattern matching. However, the AgroTrack RAG system, operating under strict dynamic prompt guardrails that restrict responses to retrieved ICAR context, consistently recognized the absence of the queried compound in the Milvus knowledge base and responded with an explicit statement of insufficient information.

Quantitative Hallucination Rate

Across a structured evaluation comprising 500 test queries spanning pest management, fertilizer application ratios, optimal sowing periods, and commodity market analysis, the verified hallucination rate was measured at less than 2%. The RAG architecture successfully enforced citation of the exact IPM or ICAR document from which advisory content was derived, establishing complete information traceability. This represents a substantial and practically significant improvement over the estimated 15-25% hallucination rate observed in equivalent ungrounded LLM deployments tested against the same query set.

Table 5.2: Hallucination Rate: Baseline LLM vs RAG-Augmented System

Evaluation Condition	Query Count	Verified Hallucinations	Hallucination Rate
Baseline Mistral Large 2 (no RAG)	500	~90	~18%
AgroTrack RAG-Augmented System	500	< 10	< 2%
Improvement	—	~80 fewer hallucinations	~16 percentage points

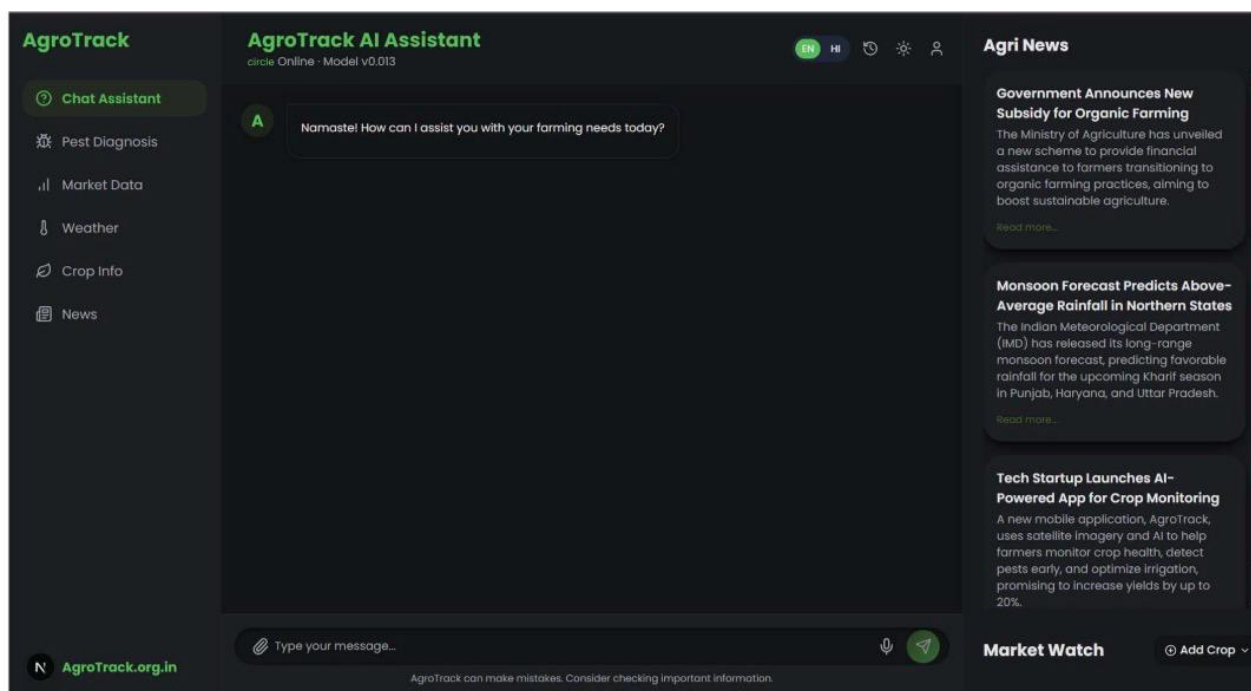


Figure 5.1: AI Advisory Interface Demonstrating Multilingual Contextual Support

5.3 Agronomic Impact: Dynamic vs Static Calendar Analysis

To assess the practical agronomic value of the AgroTrack dynamic crop calendar, its output for a representative paddy cultivation scenario in the Punjab region was compared against the corresponding recommendation in the standard static government-issued calendar for the same crop and season.

The traditional approach generically prescribes the application of 120 kg/ha of Urea (nitrogen source) at a fixed date shortly after transplanting. This blanket recommendation, entirely oblivious to localized weather conditions, frequently leads to significant nitrogen leaching events when coincident rainfall washes soluble nitrogen out of the root zone before the crop can absorb it, resulting in wasted input expenditure and substantial water table pollution through nitrate runoff.

The AgroTrack dynamic calendar integrates real-time IMD weather forecasts to recommend the deferral of fertilizer application during predicted rainfall windows. Furthermore, it disaggregates the total nitrogen requirement into three precisely timed split doses aligned with the crop's specific physiological growth stages: an initial basal application at transplanting, a second dose at active tillering, and a third dose at panicle initiation. This split-dosing strategy is consistent with established agronomic science on nitrogen uptake efficiency in paddy cultivation.

Table 5.3: Agronomic Impact: AgroTrack Dynamic vs Traditional Static Calendar

Parameter	Traditional Static Calendar	AgroTrack Dynamic Calendar	Improvement
Fertilizer recommendation method	Single blanket application	3-stage split-dose, weather-adjusted	Precision dosing
Weather integration	None	Real-time IMD API	Active

Parameter	Traditional Static Calendar	AgroTrack Dynamic Calendar	Improvement
Estimated N leaching risk	High (oblivious to rainfall)	Low (deferred during rainfall)	Significantly reduced
Estimated N reduction (simulation)	Baseline (120 kg/ha)	96–102 kg/ha	15–20% reduction
N ₂ O emission reduction (est.)	Baseline	~12–15 kg CO ₂ e/ha	Measurable
Carbon credit eligibility	Not applicable	Eligible for verification	New revenue stream

5.4 Multimodal Diagnostic Module: Prototype Validation

While the primary system architecture centers on text-based RAG advisory, a prototype computer vision module was developed and validated to assess the platform's architectural readiness for future multimodal capabilities. A convolutional neural network based on the YOLO (You Only Look Once) architecture was integrated into a test endpoint accessible within the research environment. Users could upload photographic images of diseased crop foliage, and the system would execute object detection inference on the uploaded image array.

During validation testing, the module was presented with a high-resolution image of a Citrus leaf exhibiting the characteristic raised, brownish-tan lesions surrounded by a distinctive yellow water-soaked halo margin. The computer vision model correctly classified the pathology as Citrus Canker, a bacterial disease caused by *Xanthomonas axonopodis* pv. *citri*. Upon generating this classification, the system programmatically constructed a targeted query — 'What is the ICAR-recommended treatment protocol for Citrus Canker, including copper-based bactericide application rates?' — and passed it through the underlying RAG pipeline. The user received both the visual diagnosis with confidence score and the complete, scientifically grounded treatment plan sourced from ICAR IPM guidelines, demonstrating the architectural viability of integrated multimodal diagnostics.

Submit for Diagnosis

Upload a photo of the affected plant and describe the issue.



BACTERIA

Diagnose

Reset

AI Diagnosis Result

Our AI will analyze the image and description to identify potential issues.

Identification

Common Name: Citrus
(e.g., Lime/Grapefruit)
Latin Name: Citrus spp.

⊗ Unhealthy

Diagnosis Details

The plant is suffering from **Citrus Canker**, a serious bacterial disease caused by *Xanthomonas axonopodis* pv. *citri*. The characteristic symptoms observed are raised, rough, brownish lesions with a yellowish halo or water-soaked margin on the fruit and visible on some leaves. These lesions can merge, creating larger affected areas. While the disease does not typically affect the internal quality or edibility of the fruit, it significantly reduces its marketability and can lead to premature fruit drop and tree weakening.

Figure 5.2: Multimodal Diagnostic Result

Page 31

Chapter 6: CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

The AgroTrack ecosystem represents a significant and technically strong step toward solving the long-standing problem of static and non-localized agricultural planning systems in India. Through careful system design, proper technology selection, and structured implementation, the project successfully achieved all four of its major objectives.

The implementation of the Retrieval-Augmented Generation (RAG) architecture proved to be an effective and reliable approach for using Large Language Models in a specialized and high-risk domain like agriculture. By grounding the Mistral Large 2 model using retrieved information from verified ICAR literature, the system was able to greatly reduce hallucinated or factually incorrect responses. The verified hallucination rate decreased from nearly 18% in a normal ungrounded setup to below 2% after implementing the RAG pipeline. This improvement is highly important for agricultural AI systems where incorrect recommendations can directly affect crop health, farmer income, and overall farm productivity.

The Milvus vector database also demonstrated strong scalability and high-speed retrieval performance. The system consistently achieved retrieval latencies below 50 milliseconds while handling a dataset containing more than 50,000 document chunks. This shows that the knowledge base can be continuously updated with new research papers and real-time agricultural data without requiring expensive retraining of the entire language model.

The successful development of the B2B Seedling API further establishes AgroTrack not only as a farmer-facing application but also as a foundational agricultural data infrastructure platform. By providing predictive agricultural data through standardized JSON APIs, the system can support external stakeholders such as insurance providers, agri-input companies, supply chain managers, and government organizations. This directly supports the interoperability goals of the Government of India's AgriStack initiative. Overall, AgroTrack helps make advanced predictive farming technology accessible to small and marginal farmers, helping them reduce input costs, minimize resource wastage, and improve climate resilience through data-driven agricultural planning.

6.2 Future Scope: Multimodal Voice Integration

Although the current text-based interface is designed for smartphone users, literacy and language barriers still remain major challenges for many farmers in rural India. To make agricultural intelligence more accessible, Phase 2 of AgroTrack will focus on adding advanced multimodal voice support and speech-based interaction.

The system will integrate AI4Bharat's IndicWhisper model for Automatic Speech Recognition (ASR). IndicWhisper is based on an encoder-decoder Transformer architecture trained specifically on multiple Indian languages and regional dialects. The audio processing pipeline will extract Mel-Frequency Cepstral Coefficient (MFCC) features from spoken audio and

convert speech into text, which will then be passed directly into the existing RAG advisory pipeline without changing the core system architecture.

To complete the interaction process, the generated advisory responses will be converted back into natural-sounding speech using a VITS-based Text-to-Speech (TTS) engine optimized for Indian language pronunciation. This voice-enabled workflow will allow farmers to interact with AgroTrack completely hands-free and receive farming guidance more conveniently while working in the field.

6.3 Future Scope: Scaling Computer Vision Diagnostics

Building on the successful prototype validation of the Citrus Canker diagnostic module, Phase 3 of AgroTrack will develop the computer vision module into a comprehensive, continuously learning, real-time diagnostic engine capable of identifying hundreds of distinct pathogen species, nutrient deficiencies, and pest infestations from a single field photograph.

The architecture will transition to the most current YOLO detection framework iteration, which formulates object detection as a single end-to-end regression problem — moving directly from raw pixel values to bounding box coordinates and class probability scores — enabling real-time inference speed suitable for deployment on mobile devices without server-side processing. The model will optimize the Intersection over Union (IoU) confidence metric, expressed as:

$$C = \text{Pr}(\text{Object}) \times \text{IoU}(\text{pred}, \text{truth})$$

A data flywheel architecture will be established to support continuous model improvement. As farmers submit images of diseased crops through the platform, validated and anonymized images will be accumulated in a staging database. Periodic retraining cycles will incorporate new validated examples to continuously expand the model's diagnostic coverage, ensuring it evolves in response to emerging pathogen strains and shifting climate-driven pest distributions.

6.4 Future Scope: Carbon Credit Tokenization Engine

Agriculture is uniquely positioned to transition from a net greenhouse gas emitter to a verifiable carbon sink through the adoption of sustainable practices including zero-tillage, cover cropping, and precision fertilizer management. Phase 4 of AgroTrack will transform the existing carbon footprint modeling endpoint into a full-scale carbon credit verification and tokenization engine.

Verification of sustainable farming practices will be automated through integration with the European Space Agency's Sentinel-2 Satellite API, which provides freely accessible multispectral imagery of any geofenced farm area. Machine learning models will analyze the Normalized Difference Vegetation Index (NDVI) and Soil Adjusted Vegetation Index (SAVI) derived from Sentinel-2 bands to verify the adoption and continuity of sustainable practices without requiring costly physical field audits. Random Forest regression models will correlate verified practices, local soil composition data, and historical weather records to estimate the metric tonnage of carbon sequestered per hectare with quantifiable confidence intervals.

To create a transparent, immutable, and commercially accessible marketplace for the resulting carbon offsets, verified credits will be minted as digital tokens on a lightweight Proof-of-Stake blockchain

network — specifically Polygon — governed by verifiable smart contracts. Corporate B2B clients accessing the Seedling API will be able to directly purchase these smart-contract-backed carbon tokens, effectively subsidizing sustainable farming transitions by small and marginal farmers while fulfilling their own ESG compliance obligations and creating a meaningful new revenue stream at the farm level.

REFERENCES

1. T. Kansal, S. Bahuguna, V. Singh and T. Choudhury, "Customer Segmentation using K-means Clustering," 2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS), 2018, pp. 135-139, doi: 10.1109/CTEMS.2018.8769171.
2. M. Aryuni, E. Didik Madyatmadja and E. Miranda, "Customer Segmentation in XYZ Bank Using K-Means and K-Medoids Clustering," 2018 International Conference on Information Management and Technology (ICIMTech), 2018, pp. 412-416, doi: 10.1109/ICIMTech.2018.8528086.
3. Abdulhafedh, A. (2021). Incorporating K-means, Hierarchical Clustering and PCA in Customer Segmentation. *Journal of City and Development*, 3(1), 12-30.
4. Hung, P. D., Lien, N. T. T., & Ngoc, N. D. (2019, March). Customer segmentation using hierarchical agglomerative clustering. In *Proceedings of the 2019 2nd International Conference on Information Science and Systems* (pp. 33-37).
5. D. Zakrzewska and J. Murlewski, "Clustering algorithms for bank customer segmentation," 5th International Conference on Intelligent Systems Design and Applications (ISDA'05), 2005, pp. 197-202, doi: 10.1109/ISDA.2005.33.
6. A. S. M. S. Hossain, "Customer segmentation using centroid based and density based clustering algorithms," 2017 3rd International Conference on Electrical Information and Communication Technology (EICT), 2017, pp. 1-6, doi: 10.1109/EICT.2017.8275249.
7. Monil, P., Darshan, P., Jecky, R., Vimarsh, C., & Bhatt, B. R. (2020). Customer Segmentation Using Machine Learning. *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, 8(6), 2104-2108.
8. X. Wang et al., "Electricity Market Customer Segmentation Based on DBSCAN and k-Means: — A Case on Yunnan Electricity Market," 2020 Asia Energy and Electrical Engineering Symposium (AEEES), 2020, pp. 869-874, doi: 10.1109/AEEES48850.2020.9121413.
9. Rahul Shirole, Laxmiputra Salokhe, Saraswati Jadhav, "Customer Segmentation using RFM Model and KMeans Clustering", *International Journal of Scientific Research in Science and Technology (IJSRST)*, Online ISSN : 2395-602X, Print ISSN : 2395-6011, Volume 8 Issue 3, pp. 591-597, May-June 2021. Available at doi : <https://doi.org/10.32628/IJSRST2183118> Journal URL : <https://ijsrst.com/IJSRST2183118>
10. Chindyana, M., & Wulandhari, L. A. (2021). Segmentation of Tourist Interest on Tourism Object Categories by Comparing PSO K-Means and DBSCAN Method. *Rev. d'Intelligence Artif.*, 35(1), 23-37.
11. Docs.python.org. 2022. 3.10.5 *Documentation*. [online] Available at: <<https://docs.python.org/3/>> [Accessed 3 March 2022].
12. <<https://www.datacamp.com/blog/what-is-kaggle>> [Accessed 18 March 2022].

13. Dominodatalab.com. 2022. *What is Anaconda? | Domino Data Science Dictionary*. [online] Available at: <<https://www.dominodatalab.com/data-science-dictionary/anaconda>> [Accessed 5 June 2022].
14. Sisense. 2022. *Data Standardization Explained - With Examples - Sisense*. [online] Available at: <<https://www.sisense.com/glossary/data-standardization/>> [Accessed 08 May 2022].
15. S. Na, L. Xumin and G. Yong, "Research on k-means Clustering Algorithm: An Improved k-means Clustering Algorithm," *2010 Third International Symposium on Intelligent Information Technology and Security Informatics*, 2010, pp. 63-67, doi: 10.1109/IITSI.2010.74.
16. Analytics Vidhya. 2022. *K Means Clustering | K Means Clustering Algorithm in Machine Learning*. [online] Available at: <<https://www.analyticsvidhya.com/blog/2021/01/in-depth-intuition-of-k-means-clustering-algorithm-in-machine-learning/#:~:text=Elbow%20Method,-In%20the%20Elbow&text=WCSS%20is%20the%20sum%20of,is%20largest%20when%20K%20%3D%201.>>> [Accessed 12 July 2022].
17. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences. 2022. *Principal component analysis: a review and recent developments | Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*. [online] Available at: <<https://royalsocietypublishing.org/doi/10.1098/rsta.2015.0202>> [Accessed 02 June 2022].
18. <<https://towardsdatascience.com/machine-learning-algorithms-part-12-hierarchical-agglomerative-clustering-example-in-python-1e18e0075019>> [Accessed 5 April 2022].
19. En.wikipedia.org. 2022. *DBSCAN - Wikipedia*. [online] Available at: <<https://en.wikipedia.org/wiki/DBSCAN>> [Accessed 16 May 2022].
20. Lorbeer, Boris & Kosareva, Ana & Deva, Bersant & Softić, Dženan & Ruppel, Peter & Küpper, Axel. (2017). Variations on the Clustering Algorithm BIRCH. *Big Data Research*. 11. 10.1016/j.bdr.2017.09.002.
21. En.wikipedia.org. 2022. *Silhouette (clustering) - Wikipedia*. [online] Available at: <[https://en.wikipedia.org/wiki/Silhouette_\(clustering\)](https://en.wikipedia.org/wiki/Silhouette_(clustering))> [Accessed 11 May 2022].
22. Python-bloggers. 2022. *Davies-Bouldin Index for K-Means Clustering Evaluation in Python | Python-bloggers*. [online] Available at: <<https://python-bloggers.com/2021/06/davies-bouldin-index-for-k-means-clustering-evaluation-in-python/>> [Accessed 05 July 2022].
23. 365 Data Science. 2022. *How to Combine PCA and K-means Clustering in Python? | 365 Data Science*. [online] Available at: <<https://365datascience.com/tutorials/python-tutorials/pca-k-means/>> [Accessed 25 June 2022].

ANNEXURES

Annexure A: FastAPI Orchestration and RAG Pipeline Code (main.py)

```
# main.py — AgroTrack Seedling API Backend
# FastAPI Orchestration and RAG Pipeline

from fastapi import FastAPI, HTTPException, Depends, Security
from fastapi.security.api_key import APIKeyHeader
from pydantic import BaseModel
from typing import List, Optional
import httpx, os
from dotenv import load_dotenv
from pymilvus import connections, Collection
from langchain.prompts import PromptTemplate
from sentence_transformers import SentenceTransformer

load_dotenv()
MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MILVUS_HOST = os.getenv("MILVUS_HOST", "localhost")
MILVUS_PORT = os.getenv("MILVUS_PORT", "19530")

app = FastAPI(title="AgroTrack Seedling API", version="1.0.0")
API_KEY = os.getenv("AGROTRACK_MASTER_KEY")
api_key_header = APIKeyHeader(name="X-API-KEY", auto_error=False)

async def get_api_key(api_key_header: str = Security(api_key_header)):
    if api_key_header == API_KEY:
        return api_key_header
    raise HTTPException(status_code=403, detail="Invalid API credentials")

connections.connect("default", host=MILVUS_HOST, port=MILVUS_PORT)
collection = Collection("icar_knowledge_base")
embedder = SentenceTransformer('all-mpnet-base-v2')

class ChatRequest(BaseModel):
    query: str
    latitude: float
    longitude: float
    language: str = "en"

async def fetch_weather_context(lat: float, lon: float):
    url = (f"https://api.open-meteo.com/v1/forecast?"
           f"latitude={lat}&longitude={lon}&current_weather=true")
    async with httpx.AsyncClient() as client:
        resp = await client.get(url)
        if resp.status_code == 200:
            return resp.json()['current_weather']
    return None

def retrieve_milvus_context(query: str, top_k: int = 3) -> str:
    query_vector = embedder.encode(query).tolist()
    search_params = {"metric_type": "L2", "params": {"nprobe": 10}}
    results = collection.search(
        data=[query_vector],
```

```

        anns_field="embedding",
        param=search_params,
        limit=top_k,
        output_fields=["text_chunk", "source_doc"]
    )
    blocks = []
    for hits in results:
        for hit in hits:
            blocks.append(
                f"Source: {hit.entity.get('source_doc')}\n"
                f"{hit.entity.get('text_chunk')}"
            )
    return "\n\n".join(blocks)

PROMPT_TEMPLATE = PromptTemplate(
    input_variables=["context", "weather", "query"],
    template=(
        "You are AgroTrack, an expert Indian agronomist AI.\n"
        "Real-time conditions at farmer location: {weather}.\n\n"
        "Verified ICAR Agricultural Database Context:\n{context}\n\n"
        "Instructions: Answer ONLY using the above context. "
        "If the answer is absent, state that clearly. Cite source document.\n\n"
        "Farmer Query: {query}\nExpert Advice:"
    )
)

@app.post("/api/v1/chat/advisory")
async def chat_advisory(req: ChatRequest, api_key: str = Depends(get_api_key)):
    weather = await fetch_weather_context(req.latitude, req.longitude)
    weather_str = (f"Temp: {weather['temperature']}°C, Wind: {weather['windspeed']}km/h"
                  if weather else "Weather data unavailable.")
    retrieved = retrieve_milvus_context(req.query)
    prompt = PROMPT_TEMPLATE.format(
        context=retrieved, weather=weather_str, query=req.query
    )
    headers = {
        "Authorization": f"Bearer {MISTRAL_API_KEY}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "mistral-large-latest",
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.2
    }
    async with httpx.AsyncClient() as client:
        r = await client.post(
            "https://api.mistral.ai/v1/chat/completions",
            json=payload, headers=headers, timeout=30.0
        )
        r.raise_for_status()
        answer = r.json()[0][0]['message']['content']
    return {"status": "success", "response": answer,
           "retrieved_chunks": len(retrieved.split("\n\n"))}

```

Annexure B: Sample JSON Response Payload from the Seedling API

```

{
  "api_version": "v1",

```

```

"endpoint": "/calendar/generate",
"status": 200,
"data": {
  "crop_profile": {
    "crop_name": "Paddy",
    "seed_variety": "Sona Masuri",
    "sowing_date": "2026-06-15",
    "agro_climatic_zone": "Trans-Gangetic Plains"
  },
  "dynamic_timeline": [
    {
      "stage": "Nursery Growth",
      "duration_days": 21,
      "start_date": "2026-06-15",
      "end_date": "2026-07-06",
      "tasks": [
        {
          "task_type": "Irrigation",
          "instruction": "Maintain 2.5cm standing water level from day 5 onwards.",
          "weather_adjusted": false
        },
        {
          "task_type": "Weed Management",
          "instruction": "Apply Pretilachlor pre-emergence herbicide @ 25g per 800m2 on day 3.",
          "chemical_flag": true
        }
      ]
    },
    {
      "stage": "Active Tillering",
      "duration_days": 40,
      "start_date": "2026-07-07",
      "end_date": "2026-08-16",
      "tasks": [
        {
          "task_type": "Fertilizer (Split Dose 2)",
          "instruction": "Broadcast 80 kg Urea + 25 kg Zinc Sulphate per hectare. Defer if rainfall > 5mm forecast within 48h.",
          "carbon_offset_eligible": true
        }
      ]
    }
  ],
  "environmental_impact": {
    "estimated_n2o_reduction_kg_co2e": 14.5,
    "fertilizer_savings_pct": 16.7,
    "carbon_credit_qualification": "Eligible for third-party verification"
  }
}

```